



Constructor & destructor in inheritance ÷

Constructor ÷

```
#include <iostream>
```

```
using namespace std;
```

```
class A
```

```
{
```

```
public:
```

```
A()
```

```
{
```

```
cout << "In Hello from base class";
```

```
}
```

```
};
```

```
class B : public A
```

```
{
```

```
public:
```

```
B() : A()
```

```
{
```

```
cout << "In Hello from derived class";
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
B obj;
```

```
return 0;
```

```
}
```

firstly B will call the class A constructor then it will work

Here we doesn't need to explicitly call the class A constructor if class A constructor consist of default constructor

The compiler automatically calls it



Important :

- When a base class object is created, only the constructor & destructor of the base class are called.
- When a derived class object is created, the base class constructor is called first, followed by the derived class constructor.

A : A() {

int x;

x = 0;

};

(p) A : (p) A() {

x = d;

};

};

() A() {

(p) A() {

};

};



Program: #include <iostream>
using namespace std;

class A

{

private:

int a;

public:

A(int k)

{

a = k;

}

};

class B : public A

{

private:

int b;

public:

B(int x, int y) : A(y)

{

b = x;

}

};

int main()

{

B obj(5, 7);

return 0;

}

Destructor ÷

Program: `#include <iostream>`
`using namespace std;`

Class A

{

public:

~A()

{

cout << "In Hello from class A destructor";

}

};

Class B

{

public:

~B() → firstly the coding of B will work then after by-default
the compiler write compulsory code in B for calling base class

cout << "In Hello from class B destructor"; destructor

{

int main()

{

A obj1;

B obj2;

return 0;

}

Here: first derived class destructor
works then the base class

Child class → Parent class